Sniper Bot — Checkpoint 1

Step-by-step code replacement guide

This PDF organizes the four files from the previous Checkpoint 1 response. It is a code handoff guide, not proof that the changes are compatible with your exact project. Before running, compare imports and function signatures with your current ZIP, especially **src/api.js**, **src/paper.js**, and the Worker KV binding.

Safety: paper mode only. Do not add private keys, signing, or live trading. Keep a backup or Git commit before replacing files.

Recommended order

1. Add src/signals.js
2. Add test/signals.test.js
3. Replace src/main.js
4. Replace worker/index.js and deploy it
5. Run the verification commands and inspect JSONL logs

STEP 1 — Create `src/signals.js`

Create this new helper file. It computes signal keys, signal age, and queue gaps.

```
// Pure helpers for signal identity and queue/latency checks.
// No dependencies; safe to test with Node's built-in test runner.

export function signalKey(signal) {
  const mint = typeof signal?.mint === "string" ? signal.mint : "";
  const sig = typeof signal?.sig === "string" ? signal.sig : "";
  if (sig) return `${sig}:${mint}`;
  // Without a transaction signature, deduplication cannot be reliable.
  return `queue:${signal?.id ?? "unknown"}:${mint}`;
}

export function signalAgeMs(signal, now = Date.now()) {
  const receivedAt = Number(signal?.ts);
  if (!Number.isFinite(receivedAt) || receivedAt <= 0) return null;
  return Math.max(0, now - receivedAt);
}

export function sequenceGap(since, oldestId) {
  if (!Number.isFinite(since) || !Number.isFinite(oldestId)) return 0;
  return Math.max(0, oldestId - since - 1);
}
```

STEP 2 — Create `test/signals.test.js`

Create the test folder if needed. These use Node's built-in test runner; no npm package is required.

```
import test from "node:test";
import assert from "node:assert/strict";
import { signalAgeMs, signalKey, sequenceGap } from "../src/signals.js";

test("signal identity distinguishes transactions for the same mint", () => {
  assert.notEqual(
    signalKey({ id: 1, sig: "tx-a", mint: "mint-x" }),
    signalKey({ id: 2, sig: "tx-b", mint: "mint-x" }),
  );
});

test("signal identity deduplicates the same transaction and mint", () => {
  assert.equal(
    signalKey({ id: 1, sig: "tx-a", mint: "mint-x" }),
    signalKey({ id: 2, sig: "tx-a", mint: "mint-x" }),
  );
});

test("unsigned signals use queue identity instead of mint-only identity", () => {
  assert.notEqual(
    signalKey({ id: 1, mint: "mint-x" }),
    signalKey({ id: 2, mint: "mint-x" }),
  );
});

test("signal age is measurable and missing timestamps are explicit", () => {
  assert.equal(signalAgeMs({ ts: 1000 }, 2500), 1500);
  assert.equal(signalAgeMs({}, 2500), null);
});

test("queue gap counts IDs that could have fallen out of the bounded queue", () => {
  assert.equal(sequenceGap(10, 15), 4);
  assert.equal(sequenceGap(10, 11), 0);
});
```

STEP 3 — Replace `src/main.js`

This version processes retained signals sequentially and adds structured signal/latency logs. Verify imported function signatures before use.

```
import { existsSync } from "node:fs";
import { cfg } from "./config.js";
import { getSignals, heartbeat, notify } from "./api.js";
import { runGate } from "./gate.js";
import { openPaper, tick, openCount, isOpen } from "./paper.js";
import { log } from "./log.js";
import { signalAgeMs, signalKey, sequenceGap } from "./signals.js";

// Inspect retained backlog at startup. Old signals are logged, not traded.
let since = 0;
const seen = new Set();

const configuredAge = Number(process.env.MAX_SIGNAL_AGE_MS);
const maxSignalAgeMs =
  process.env.MAX_SIGNAL_AGE_MS && Number.isFinite(configuredAge)
    ? Math.max(0, configuredAge)
    : 60_000;

async function handle(s) {
  const key = signalKey(s);
  const receivedAgeMs = signalAgeMs(s);

  log("signal_received", {
    signalId: s.id,
    signalKey: key,
    mint: s.mint,
    sig: s.sig || null,
    receivedAgeMs,
  });

  if (receivedAgeMs === null) {
    log("signal_skip", {
      signalId: s.id,
      signalKey: key,
      mint: s.mint,
      reason: "missing_relay_timestamp",
    });
    return;
  }

  if (receivedAgeMs > maxSignalAgeMs) {
    log("signal_skip", {
      signalId: s.id,
      signalKey: key,
      mint: s.mint,
      reason: "stale_signal",
      receivedAgeMs,
      maxSignalAgeMs,
    });
    return;
  }

  if (existsSync("HALT")) {
```

```
  log("halted_skip", {
    signalId: s.id,
    signalKey: key,
    mint: s.mint,
    reason: "local_halt",
  });
  return;
}

const gateStartedAt = Date.now();
let g;

try {
  g = await runGate(s.mint, cfg.gate);
} catch (e) {
  log("gate_error", {
    signalId: s.id,
    signalKey: key,
    mint: s.mint,
    sig: s.sig || null,
    receivedAgeMs,
    gateMs: Date.now() - gateStartedAt,
    msg: String(e),
  });
  return;
}

const gateMs = Date.now() - gateStartedAt;
const endToGateMs = signalAgeMs(s);

log("gate", {
  signalId: s.id,
  signalKey: key,
  mint: s.mint,
  sig: s.sig || null,
  pass: g.pass,
  reasons: g.reasons,
  top10Pct: g.top10Pct,
  receivedAgeMs: endToGateMs,
  gateMs,
});

if (!g.pass) return;

if (isOpen(s.mint)) {
  log("signal_skip", {
    signalId: s.id,
    signalKey: key,
    mint: s.mint,
    reason: "position_already_open",
  });
  return;
}

const entryStartedAt = Date.now();

await openPaper(s.mint, s.sig, {
  signalId: s.id,
```

```
      signalKey: key,
      receivedAgeMs: endToGateMs,
      gateMs,
    });

    log("entry_attempt_complete", {
      signalId: s.id,
      signalKey: key,
      mint: s.mint,
      entryAttemptMs: Date.now() - entryStartedAt,
      totalRelayToEntryMs: signalAgeMs(s),
    });
  }

  async function pollOnce() {
    const {
      signals = [],
      seq = since,
      oldestId = null,
      halt = false,
    } = await getSignals(since);

    if (oldestId !== null) {
      const gap = sequenceGap(since, oldestId);
      if (gap > 0) {
        log("signal_gap", {
          since,
          oldestId,
          missingSequenceCount: gap,
        });
      }
    }

    since = seq;
    const halted = halt || existsSync("HALT");

    for (const s of signals) {
      const key = signalKey(s);

      if (seen.has(key)) {
        log("signal_duplicate", {
          signalId: s.id,
          signalKey: key,
          mint: s.mint,
          sig: s.sig || null,
        });
        continue;
      }

      seen.add(key);

      if (halted) {
        log("halted_skip", {
          signalId: s.id,
          signalKey: key,
          mint: s.mint,
          reason: "halt_active_at_poll",
        });
```

```
      continue;
    }

    // Sequential processing prevents concurrent gate checks from racing MAX_OPEN.
    try {
      await handle(s);
    } catch (e) {
      log("error", {
        where: "handle",
        signalId: s.id,
        signalKey: key,
        mint: s.mint,
        msg: String(e),
      });
    }
  }
}

async function pollLoop() {
  try {
    await pollOnce();
  } catch (e) {
    log("error", { where: "poll", msg: String(e) });
  }

  setTimeout(pollLoop, cfg.pollMs);
}

log("start", {
  pollMs: cfg.pollMs,
  paperOnly: true,
  startupBacklog: "inspect_retained_queue",
  maxSignalAgeMs,
});

notify("Bot started (paper mode)");
pollLoop();

setInterval(
  () => tick().catch((e) => log("error", { where: "tick", msg: String(e) })),
  cfg.priceMs,
);

setInterval(() => heartbeat(openCount()), cfg.heartbeatMs);
heartbeat(openCount());

process.on("SIGINT", () => {
  log("stop", {});
  process.exit(0);
});
```

STEP 4 — Replace `worker/index.js`

This updates the queue response with oldestId and uses transaction-signature + mint deduplication. Cloudflare KV read-modify-write races remain possible.

```
const WSOL = "So11111111111111111111111111111111111111112";
const USDC = "EPjFWdd5AufqSSqeM2qN1xzybapC8G4wEGGkZwyTDt1v";
const USDT = "Es9vMFrzaCERmJfrF4H2FYD4KCoNkY11McCe8BenwNYB";
const IGNORE = new Set([WSOL, USDC, USDT]);
const QMAX = 50;
const HB_LATE_MS = 12 * 60 * 1000;

const json = (o, status = 200) =>
  new Response(JSON.stringify(o), {
    status,
    headers: { "content-type": "application/json" },
  });

async function tg(env, text) {
  if (!env.TG_TOKEN || !env.TG_CHAT) return;

  await fetch(`https://api.telegram.org/bot${env.TG_TOKEN}/sendMessage`, {
    method: "POST",
    headers: { "content-type": "application/json" },
    body: JSON.stringify({
      chat_id: env.TG_CHAT,
      text: String(text).slice(0, 3500),
    }),
  });
}

// Adapt this after verifying actual Helius webhook payloads.
function extractMints(tx) {
  const out = new Set();

  for (const t of tx.tokenTransfers ?? []) {
    if (t.mint && !IGNORE.has(t.mint)) out.add(t.mint);
  }

  return [...out];
}

const readQ = async (env) =>
  (await env.KV.get("queue", "json")) ?? {
    seq: 0,
    halt: false,
    items: [],
  };

export default {
  async fetch(req, env) {
    const url = new URL(req.url);
    const path = url.pathname;

    if (path === "/hook" && req.method === "POST") {
      if (req.headers.get("authorization") !== env.HELIUS_AUTH) {
        return json({ error: "unauthorized" }, 401);
      }
```

```
  const body = await req.json().catch(() => []);
  const txs = Array.isArray(body) ? body : [body];
  const q = await readQ(env);
  let changed = false;

  for (const tx of txs) {
    const receivedAt = Date.now();
    const sig = typeof tx?.signature === "string" ? tx.signature : "";

    for (const mint of extractMints(tx)) {
      // Deduplicate a transaction/mint pair, not all future events for a mint.
      // Missing signatures cannot be reliably deduplicated across retries.
      const duplicate =
        sig && q.items.some((i) => i.sig === sig && i.mint === mint);

      if (duplicate) continue;

      q.seq += 1;
      q.items.push({
        id: q.seq,
        mint,
        sig,
        ts: receivedAt,
      });
      changed = true;
    }
  }

  if (changed) {
    q.items = q.items.slice(-QMAX);
    await env.KV.put("queue", JSON.stringify(q));
  }

  return json({ ok: true });
}

if (req.headers.get("x-key") !== env.APP_KEY) {
  return json({ error: "unauthorized" }, 401);
}

if (path === "/signals" && req.method === "GET") {
  const since = Number(url.searchParams.get("since") ?? -1);
  const q = await readQ(env);

  // since=0 requests retained backlog; the phone filters stale entries.
  const signals =
    since < 0 ? [] : q.items.filter((i) => i.id > since);

  const oldestId = q.items.length ? q.items[0].id : q.seq + 1;

  return json({
    signals,
    seq: q.seq,
    oldestId,
    halt: q.halt,
  });
}

if (path === "/heartbeat" && req.method === "POST") {
```

```
      const b = await req.json().catch(() => ({}));

      await env.KV.put(
        "hb",
        JSON.stringify({ ts: Date.now(), open: b.open ?? 0 }),
      );

      return json({ ok: true });
    }

    if (path === "/notify" && req.method === "POST") {
      const b = await req.json().catch(() => ({}));
      await tg(env, b.text ?? "");
      return json({ ok: true });
    }

    if (path === "/halt" && req.method === "POST") {
      const q = await readQ(env);
      q.halt = url.searchParams.get("on") === "1";

      await env.KV.put("queue", JSON.stringify(q));
      await tg(env, q.halt ? "HALT set: no new entries" : "HALT cleared");

      return json({ halt: q.halt });
    }

    return json({ error: "not found" }, 404);
  },

  async scheduled(_event, env) {
    const hb = await env.KV.get("hb", "json");
    if (!hb) return;

    const late = Date.now() - hb.ts > HB_LATE_MS;
    const alerted = await env.KV.get("alerted");

    if (late && !alerted) {
      const mins = Math.round((Date.now() - hb.ts) / 60000);

      await tg(
        env,
        `Phone silent for ${mins} min. Open paper positions at last heartbeat: ${hb.open}`,
      );

      await env.KV.put("alerted", "1", { expirationTtl: 3600 });
    } else if (!late && alerted) {
      await env.KV.delete("alerted");
    }
  },
};
```

STEP 5 — Verify on Android / Termux

After copying the files into the project, run:

```
cd ~/sniper-bot
git status --short
node --test test/*.test.js
node --check src/main.js
node --check src/signals.js
node --check worker/index.js
```

If checks pass, commit the changes from your Git client or shell:

```
git add worker/index.js src/main.js src/signals.js test/signals.test.js
git commit -m "fix(paper): improve signal integrity logging"
git push
```

Then deploy the updated Worker from Cloudflare and start the paper bot:

```
cd ~/sniper-bot
./run.sh
tail -f logs/paper.jsonl
```

Expected log types include signal_received, gate, gate_error, signal_gap, signal_skip, and entry_attempt_complete. The Worker timestamp is its processing time, not necessarily the on-chain transaction time. entryAttemptMs is not a real exchange fill metric.

Compatibility warning

The previous response supplied these files without executing them against the ZIP. The current project must be checked before replacement: the supplied main.js calls openPaper with an additional metadata argument and imports isOpen, and the Worker expects a KV binding named KV. Confirm these match your source/configuration. Do not treat the PDF as a successful build or deploy result.